

Authorization (BOLA) Assessment

Self-Hosted BookStack Instance

Prepared by Kartik Pradhan

1. Executive Summary

This engagement looked at whether BookStack's object-level page permissions actually hold up when a page is accessed through its stable, ID-based redirect route (/link/{id}) instead of the normal slug-based navigation. This is a common vulnerability class, Broken Object-Level Authorization (BOLA), and it shows up in apps that expose both a human-readable path and a direct, enumerable ID for the same object.

I built a custom Python tool for this rather than relying on manual browser testing alone, so the same check could be repeated consistently instead of depending on me remembering exactly what I clicked last time.

With the tested configuration (Viewer role's read access explicitly revoked on the page), no authorization bypass turned up. The low-privilege session got an HTTP 404 with no page content. I also validated that the tool can actually catch a bypass when one exists, using a controlled test described in Section 5, so this isn't just a tool that always says everything's fine.

2. Scope

Target: Self-hosted BookStack instance (Docker: linuxserver/bookstack + linuxserver/mariadb), local test environment

Object tested: One Page, contained within one Book ("AppSec"), page ID 2

Access vector tested: GET requests via BookStack's ID-based redirect route: /link/{id}

Identities used: Owner session (Admin role, full access) and Attacker session (Viewer role, restricted access)

This assessment was intentionally scoped to read-level (GET) object-level authorization on a single object and access route, to allow the testing methodology and tooling to be properly validated before extending coverage. Section 7 details what was not covered and is proposed as follow-on work.

3. Methodology

Two authenticated sessions were established against the target application using session cookies obtained from two separate, isolated browser contexts (to avoid cookie-jar collision between accounts): an Owner session with legitimate access to the test object, and an Attacker session representing a low-privilege user without an explicit grant to that object.

For each test case, both sessions requested the same URL. The Owner's response established a baseline, confirming the object exists and giving us a distinctive content marker unique to that page. The Attacker's response was then checked for that same marker. If it showed up in the Attacker's response, that meant confirmed unauthorized access to the object, a genuine BOLA. If it didn't (for example, the Attacker got redirected to a login or error page), access control was working as it should.

This baseline/marker comparison approach was used specifically to avoid a common false-negative failure mode in authorization testing: checking only the HTTP status code is not sufficient, since some applications return HTTP 200 with an empty or generic error-shaped body even when access is correctly denied.

4. Tooling

A Python command-line tool was written for this assessment (source: bola_scanner.py). It accepts a JSON configuration file specifying the target base URL, authentication cookies for both the Owner and Attacker identities, and a list of endpoints to test, each with a path and an expected content marker.

Before running any authorization test, the tool checks that both sessions are actually authenticated (it requests an authenticated-only endpoint and confirms a successful response), rather than assuming the cookies it was given are still valid. I added this after running into a stale session cookie during development that quietly produced a misleading result. The tool is meant to fail loudly rather than report a false clean result.

For each configured endpoint, the tool requests the object as both identities, compares the responses against the baseline/marker logic described in Section 3, and prints a verdict of OK, FINDING, or SETUP ISSUE (the last indicating the Owner itself could not access the object, which signals a test configuration problem rather than an application security issue).

5. Test Execution

5.1 Positive Control Validation

Purpose: to confirm the tool correctly detects unauthorized access when it genuinely exists, rather than defaulting to a clean result regardless of actual configuration.

Configuration: The Viewer role was explicitly granted View permission on the test page.

Result: The Attacker session successfully retrieved the page (HTTP 200), and the content marker showed up in its response. This was expected and correct: the Viewer role held a genuine grant at the time of the test, so this counts as legitimate access, not a bypass.

This step is a validation of the tool, not a finding against the application. It confirms the detection logic actually works before relying on it in Section 5.2.

5.2 Restricted Configuration Assessment

Configuration: The Viewer role's View permission was explicitly revoked on the test page via BookStack's custom permissions feature. Manual verification confirmed the Viewer account could not see or navigate to the page through the normal application UI.

Result: The Owner session retrieved the page normally (HTTP 200, marker present). The Attacker session, requesting the identical object via /link/2, received an HTTP 404 response with no page content and no marker present.

Verdict: Access control was correctly enforced. No object-level authorization bypass was identified for this endpoint under this configuration.

Test Case	Configuration	Owner Result	Attacker Result	Verdict
5.1 Positive control	Viewer granted access	200, marker present	200, marker present	Detection confirmed
5.2 Restricted config	Viewer denied access	200, marker present	404, no marker	Access control held

6. Observations

The Attacker session got an HTTP 404 response instead of an HTTP 403 Forbidden when access was denied. That's worth calling out, and it's a good sign: a 403 would tell an unauthorized party that an object with that ID exists at all, even if they can't see its contents, which is its own kind of information leak. A 404 gives nothing away either way. Based on the one object tested, BookStack seems to follow this practice on this route.

7. Scope Limitations and Recommended Follow-On Work

- Single object tested. This assessment validated the methodology against one page under two permission configurations, not a broad inventory of objects. Testing more pages, chapters, and books, especially to see how permission inheritance behaves between them, would give broader assurance.
- Read-level access only. Only GET requests were tested. Function-level authorization on write operations (page edit, delete) was scoped out of this engagement, primarily because BookStack's write routes require a valid CSRF token to be captured and replayed alongside the session cookie, which the current tooling does not yet implement. This is the primary recommended next step, as broken authorization on write/delete operations is typically a higher-impact finding than read-only disclosure.
- Single access vector tested. Only the `/link/{id}` route was assessed. BookStack's API (where present) and any other ID-referencing internal routes were not tested.
- Local, non-production environment. Findings reflect a self-hosted test instance and should not be assumed to generalise to a production deployment or hosted BookStack instance without separate verification.

8. Conclusion

Within the scope tested, BookStack's object-level authorization held up for the assessed page and access route, and the choice to return 404 instead of 403 is a sensible security decision. I validated the tool against a known-positive case before trusting the negative result in Section 5.2, so I'm reasonably confident in it. The methodology and tooling here should extend fairly easily to the broader coverage and write-access testing described in Section 7.